SOTA Agents for Software Engineering

Comprehensive Literature Review and Agent Role Taxonomy

1. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework (A1)

Year: 2023/2024

Goals

- Encode software-team workflows as multi-agent SOPs to reduce cascading hallucinations.
- Produce coherent artifacts (PRD → design → code → tests) from high-level requirements.

Methods / Approach

- SOP-encoded stage boundaries with artifact hand-offs.
- Explicit role profiles (PM, Architect, PMgr, Engineer, QA).
- Execution/feedback loop to improve code quality.

Key Contributions

- Company-style multi-agent framework with SOP coordination.
- Verbatim role specialization for SDLC hand-offs.
- Open-source implementation with examples.

Identified Agent Roles

Role	**Responsibilities**
Product Manager	Elicits requirements; drafts PRD.
Architect	Designs system architecture & interfaces.
Project Manager	Coordinates milestones & hand-offs.
Engineer	Implements code per design.
QA Engineer	Designs/executes tests; validates requirements.

Role-Taxonomy Placement

Category	**Taxonomy Role**
Orchestration	Project Coordinator (SOP)
Analysis & Planning	Requirement Analyst / PM
Design	Architect / Designer
Implementation	Coder / Implementer
Quality	Tester / Reviewer
Knowledge	Documentation Writer

---

2. Magentic-One (AutoGen) (A2)

Year: 2024/2025

Goals

- Generalist multi-agent system that routes complex tasks to specialized agents.
- Demonstrate manager-worker orchestration on open-ended tasks.

Methods / Approach

- Manager plans and dispatches tasks to tool-specialist agents (web, file, code).
- AgentChat protocol for inter-agent collaboration.

Key Contributions

- Practical manager/worker pattern with pluggable tools.
- Extensible orchestration over diverse tasks.

Identified Agent Roles

Role	Responsibilities
Manager / Router	Plans & routes subtasks to specialists.
Specialist Agents	Web/file/code tools as needed.

Role-Taxonomy Placement

Category	Taxonomy Role
Orchestration	Manager / Router
Operations	Tooling / Executor

---

3. Self-Organized Agents: Ultra Large-Scale Code Generation and Optimization (A3)

Year: 2024

Goals

- Scale LLM-based code generation with self-organization among agents.

Methods / Approach

- Self-organized role emergence; large-scale code optimization workflow.

Key Contributions

- Demonstrates ultra-large-scale coordination patterns.

Identified Agent Roles

Role	Responsibilities
N/A	Roles not explicitly enumerated in the abstract/summary.

Role-Taxonomy Placement

Category	Taxonomy Role
Orchestration	Self-organized workflow (conceptual)

---

4. MapCoder: Multi-Agent Code Generation for Competitive Problem Solving (A4)

Year: 2024

Goals

- Improve code synthesis for algorithmic problems using multi-role agents.

Methods / Approach

- Pipeline with Example-Recall → Planner → Coder → Debugger.

Key Contributions

- Demonstrates benefit of explicit plan-implement-debug workflow for programming tasks.

Identified Agent Roles

Role	Responsibilities
Example-Recall	Fetches relevant examples/templates.
Planner	Outlines solution steps.
Coder	Implements solution code.
Debugger	Fixes issues via test feedback.

Role-Taxonomy Placement

Category	Taxonomy Role
Analysis & Planning	Planner / Decomposer
Implementation	Coder / Implementer
Quality	Debugger

---

5. A Taxonomy for Autonomous LLM-Powered Multi-Agent Architectures (Händler) (A5)

Year: 2024

Goals

- Provide a multi-dimensional taxonomy balancing autonomy and alignment.

Methods / Approach

- Conceptual dimensions: goal mgmt, composition, collaboration, context.

Key Contributions

- Foundational terminology and axes for analyzing multi-agent systems.

Identified Agent Roles

Role	Responsibilities
N/A	Taxonomy paper; does not define a fixed role set.

Role-Taxonomy Placement

Category	Taxonomy Role
Meta	Architecture taxonomy (reference)

---

6. CodeAgent: Autonomous Communicative Agents for Code Review (B6)

Year: 2024

Goals

- Automate code review with supervisor and reviewer agents.

Methods / Approach

- Supervisor-directed multi-agent dialogue; static/diff analysis tools.

Key Contributions

- Reports strong performance on code-review tasks.

Identified Agent Roles

Role	**Responsibilities**
Supervisor / QA-Checker	Oversees review process.
Reviewer	Performs targeted checks & suggests fixes.

Role-Taxonomy Placement

Category	**Taxonomy Role**
Orchestration	Supervisor / Critic
Quality	Reviewer

---

7. Autonomous Agents in Software Engineering: A Multi-Agent LLM Approach (B7)

Year: 2025

Goals

- Present a conceptual MA-SE framework and case studies.

Methods / Approach

- Position/architecture paper.

Key Contributions

- Unifies concepts for MA-SE; motivates evaluations.

Identified Agent Roles

Role	Responsibilities
N/A	Roles to be extracted from full text.

Role-Taxonomy Placement

Category	Taxonomy Role
Meta	Conceptual MA-SE architecture

---

8. WorldCoder, a Model-Based LLM Agent (B8)

Year: 2024

Goals

- Unify planning and acting by writing code as a world model.

Methods / Approach

- Single-agent planning via executable Python programs.

Key Contributions

- Executable world-model paradigm for reasoning/acting.

Identified Agent Roles

Role	Responsibilities
Planner+Coder (single agent)	Fused planning & coding.

Role-Taxonomy Placement

Category	Taxonomy Role
Implementation	World-Model Coder
Operations	Tooling / Executor

---

9. Tree-of-Code: A Hybrid Approach for Robust Complex Task Planning and Execution (B9)

Year: 2024

Goals

- Improve robustness of complex task solving via code-execution trees.

Methods / Approach

- Single-agent grows a tree of executable code with selection/voting.

Key Contributions

- Demonstrates gains vs CodeAct in reported datasets.

Identified Agent Roles

Role	Responsibilities
Planner/Executor (single agent)	Explores & selects executable branches.

Role-Taxonomy Placement

Category	Taxonomy Role
Analysis & Planning	Planner / Decomposer
Operations	Tooling / Executor

---

10. From LLMs to LLM-based Agents for Software Engineering: A Survey (B10)

Year: 2024

Goals

- Survey the use of LLM agents across the SDLC.

Methods / Approach

- Literature review and synthesis.

Key Contributions

- Identifies opportunities and challenges across SE tasks.

Identified Agent Roles

Role	Responsibilities
N/A	Survey paper.

Role-Taxonomy Placement

Category	Taxonomy Role
Meta	Survey (SE agent landscape)

---

11. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead (C11)

Year: 2024

Goals

- Synthesize MAS in SE; outline vision and roadmap.

Methods / Approach

- Literature review; perspective.

Key Contributions

- Vision and gaps; taxonomy pointers.

Identified Agent Roles

Role	Responsibilities
N/A	Survey paper.

Role-Taxonomy Placement

Category	Taxonomy Role
Meta	Survey (MAS in SE)

---

12. AI Agents: Evolution, Architecture, and Real-World Applications (Kapoor et al.) (C12)

Year: 2025

Goals

- Broad agent architecture review across domains.

Methods / Approach

- Survey + architecture analysis.

Key Contributions

- Architecture patterns and real-world applications.

Identified Agent Roles

Role	Responsibilities
N/A	Broad survey.

Role-Taxonomy Placement

Category	Taxonomy Role
Meta	Survey (cross-domain)

---

13. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity (METR/METER) (C13)

Year: 2025

Goals

- Quantify impact of AI on OSS developer productivity.

Methods / Approach

- Empirical study design and analysis.

Key Contributions

- Productivity measurements & caveats.

Identified Agent Roles

Role	Responsibilities
N/A	Measurement study.

Role-Taxonomy Placement

Category	Taxonomy Role
Meta	Empirical measurement

---

14. Coding with AI Assistants: Faster Performance, Bigger Flaws (Apiiro) (C14)

Year: 2025

Goals

- Assess risk/performance trade-offs of AI coding assistants.

Methods / Approach

- Industry report; large dataset claims.

Key Contributions

- Shows velocity↑ and vulnerability↑ patterns.

Identified Agent Roles

Role	Responsibilities
N/A	Industry report; not an agent system.

Role-Taxonomy Placement

Category	Taxonomy Role
Meta	Risk/Performance analysis

---

15. Agentless: Demystifying LLM-based Software Engineering Agents (C15)

Year: 2024

Goals

- Discuss capabilities/limits of SE agents; clarify claims.

Methods / Approach

- Study/discussion.

Key Contributions

- Clarifies pitfalls and realistic expectations.

Identified Agent Roles

Role	Responsibilities
N/A	Study/discussion.

Role-Taxonomy Placement

Category	Taxonomy Role
Meta	Critical discussion

---

16. The Design Space of LLM-Based AI Coding Assistants (Lau et al., VL/HCC 2025) (C16)

Year: 2024/2025

Goals

- Organize UX/interaction patterns for coding assistants.

Methods / Approach

- Design-space analysis across 90+ tools.

Key Contributions

- Taxonomy of assistant interactions; implications.

Identified Agent Roles

Role	Responsibilities
N/A	Design-space paper (not agent roles).

Role-Taxonomy Placement

Category	Taxonomy Role
Meta	Design space / taxonomy

---

17. Autonomous Agents for Software Testing (Wang et al.) (C17)

Year: 2024

Goals

- Survey frameworks/roles for testing agents (generator/executor/oracle).

Methods / Approach

- Testing-oriented agent survey & landscape.

Key Contributions

- Clarifies testing pipelines and agent roles.

Identified Agent Roles

Role	Responsibilities
Generator	Produces tests or inputs.
Executor	Runs tests & captures outcomes.
Oracle	Decides pass/fail; compares behavior.

Role-Taxonomy Placement

Category	Taxonomy Role
Quality	Tester / Fuzzer / Oracle
Operations	Tooling / Executor

---

18. "My productivity is boosted, but …" Demystifying Users' Perception on AI Coding Assistants (C18)

Year: 2025

Goals

- Understand user perceptions of AI coding assistants.

Methods / Approach

- HCI user study.

Key Contributions

- Findings on productivity, trust, and workflows.

Identified Agent Roles

Role	Responsibilities
N/A	User study; not an agent framework.

Role-Taxonomy Placement

Category	Taxonomy Role
Meta	User perception study

---

19. PotPie (OSS): Custom engineering agents grounded in your codebase (P)

Year: 2025

Goals

- Provide configurable engineering agents grounded in a repository knowledge graph.

Methods / Approach

- Neo4j KG; IDE/CI integrations; multi-tool orchestration.

Key Contributions

- Practical OSS toolkit; user-defined roles (debugger, tester, designer, onboarding).

Identified Agent Roles

Role	Responsibilities
Debugger	Guided debugging grounded in code graph.
Tester	Test gen & running with repo context.
Designer	Assists design reviews with code context.
Onboarding Guide	Explains code structure to newcomers.
KG Curator	Maintains/queries the code knowledge graph.

Role-Taxonomy Placement

Category	Taxonomy Role
Knowledge	Knowledge-Graph Curator
Quality	Debugger / Tester
Design	Architect / Designer (assistant)
Orchestration	Tooling-aware workflow

References & Links

All clickable source URLs compiled from individual paper sections.

- A1 — MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework (2023/2024)

• arXiv (abs): https://arxiv.org/abs/2308.00352

• arXiv (HTML v7): https://arxiv.org/html/2308.00352v7

• GitHub: https://github.com/FoundationAgents/MetaGPT

- A2 — Magentic-One (AutoGen) (2024/2025)

• AutoGen User Guide: https://microsoft.github.io/autogen/dev/user-guide/agentchat-user-guide/magentic-one.html

• MSR Article: https://www.microsoft.com/en-us/research/articles/magentic-one-a-generalist-multi-agent-system-for-solving-complex-tasks/

- A3 — Self-Organized Agents: Ultra Large-Scale Code Generation and Optimization (2024)

• arXiv: https://arxiv.org/abs/2404.02183

• GitHub: https://github.com/tsukushiAI/self-organized-agent

- A4 — MapCoder: Multi-Agent Code Generation for Competitive Problem Solving (2024)

• arXiv: https://arxiv.org/abs/2405.11403

• ACL 2024 PDF: https://aclanthology.org/2024.acl-long.269.pdf

- A5 — A Taxonomy for Autonomous LLM-Powered Multi-Agent Architectures (Händler) (2024)

• arXiv: https://arxiv.org/abs/2310.03659

- B6 — CodeAgent: Autonomous Communicative Agents for Code Review (2024)

• arXiv: https://arxiv.org/abs/2402.02172

- B7 — Autonomous Agents in Software Engineering: A Multi-Agent LLM Approach (2025)

• PDF: https://www.researchgate.net/profile/Gregory-Talavera/publication/388834987_Autonomous_Agents_in_Software_Engineering_A_Multi-Agent_LLM_Approach/links/67a90cc4461fb56424d320f9/Autonomous-Agents-in-Software-Engineering-A-Multi-Agent-LLM-Approach.pdf

- B8 — WorldCoder, a Model-Based LLM Agent (2024)

• arXiv: https://arxiv.org/abs/2402.12275

• NeurIPS Abstract: https://proceedings.neurips.cc/paper_files/paper/2024/hash/820c61a0cd419163ccbd2c33b268816e-Abstract-Conference.html

- B9 — Tree-of-Code: A Hybrid Approach for Robust Complex Task Planning and Execution (2024)

• arXiv: https://arxiv.org/abs/2412.14212

- B10 — From LLMs to LLM-based Agents for Software Engineering: A Survey (2024)

• arXiv: https://arxiv.org/abs/2408.02479

- C11 — LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead (2024)

• arXiv: https://arxiv.org/abs/2404.04834

- C12 — AI Agents: Evolution, Architecture, and Real-World Applications (Kapoor et al.) (2025)

• arXiv: https://arxiv.org/abs/2503.12687

- C13 — Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity (METR/METER) (2025)

• arXiv: https://arxiv.org/abs/2507.09089

- C14 — Coding with AI Assistants: Faster Performance, Bigger Flaws (Apiiro) (2025)

• Report: https://apiiro.com/blog/4x-velocity-10x-vulnerabilities-ai-coding-assistants-are-shipping-more-risks/

- C15 — Agentless: Demystifying LLM-based Software Engineering Agents (2024)

• arXiv: https://arxiv.org/abs/2407.01489

- C16 — The Design Space of LLM-Based AI Coding Assistants (Lau et al., VL/HCC 2025) (2024/2025)

• PDF: https://lau.ucsd.edu/pubs/2025_analysis-of-90-genai-coding-tools_VLHCC.pdf

- C17 — Autonomous Agents for Software Testing (Wang et al.) (2024)

• arXiv: https://arxiv.org/abs/2409.09030

- C18 — “My productivity is boosted, but …” Demystifying Users’ Perception on AI Coding Assistants (2025)

• arXiv: https://arxiv.org/abs/2508.12285

- P — PotPie (OSS): Custom engineering agents grounded in your codebase (2025)

• GitHub: https://github.com/potpie-ai/potpie

• Docs: https://docs.potpie.ai/